grblHAL Fork Suite — Overview

Three coordinated forks by `stevenrwood` and Claude Code — the **ioSender** g-code sender, the **grblHAL Simulator**, and the **iMXRT1062 (Teensy 4)** firmware — each a set of focused, individually-reviewable enhancements staged as pull requests on the fork.

> **What this is.** A single map across the three repos: the goals and how the work is organised, each fork's PRs, how to build any subset with the `apply-prs` composer, and links to the per-fork proposal trackers. The detailed file/line counts and prose for each change live in those trackers (linked at the end).

1 · Goals & process

Goal. Improve the open-source grblHAL CNC stack — the Windows sender, the PC simulator that drives it, and the controller firmware — with small, self-contained features and fixes, each kept as a clean diff so the upstream maintainers can adopt, cherry-pick, or decline them piece by piece.

How the work is organised (identical model in all three repos):

- **One purpose per branch.** Every change is a `pr/<name>` branch off the fork's base, scoped to a single feature or fix.
- **An `integration` branch** merges them all — the everyday working build, where new work happens before being routed back to its owning `pr/*` branch so each stays a clean single-feature diff.
- **A proposal tracker** (`ProposedPRs.html` + PDF) documents every PR: a features-at-a-glance index, per-PR file/line counts, dependency notes, and a "how to consume" guide. It is what gets shared with the upstream maintainers.
- **An `apply-prs` composer** (see §3) builds any chosen subset on demand — resolving dependencies and reporting overlaps — so a consumer takes only the features they want.

Why forks rather than upstream PRs. The upstream maintainers are deliberately minimalist and selective, so the strategy is to *maintain the forks and stay submittable*: each branch remains a focused, ready-to-propose diff, surfaced in the tracker, without blocking on upstream acceptance.

2 · The three forks

SENDER stevenrwood/ioSender

The WPF g-code sender (fork of `terjeio/ioSender`). **27 tracked PRs.** PRs **1–8 and 24** are already integrated into the fork's `master` baseline; the selectable set is PRs **9–23, 25–27**. Headline features: configurable main page + flyouts, USB/Ethernet/simulator connection + Connect menu, Validate-controller conformance test, Machine Setup Wizard, grbl:Settings change-highlight/revert, SD+LittleFS file manager, macro manager + pre-processor, in-app Key Mappings editor, Xbox controller support, 3D-view click-to-jog, ATC tool-change provisioning, and a full 7-locale localization pass. Hard deps: 16→15, 17→15, 25→14.

SIMULATOR stevenrwood/Simulator

The PC grblHAL simulator that ioSender's Connect dialog can launch (fork of `grblHAL/Simulator`). **5 PRs**, all off pristine `master`: `pr/ymodem-socket` (file upload over the socket), `pr/sim-probe` (modelled

stock/toolsetter/spoilboard surfaces), `pr/littlefs` (on-board filesystem), `pr/persistence-reboot` (settings persistence + `$REBOOT`), and `pr/sim-view` (3D view + material-removal carve — the integration superset). Two companion fixes live in submodule forks (`core`, `Plugin_SD_card`). CMake/MinGW build → `grblHAL_sim.exe`.

FIRMWARE stevenrwood/iMXRT1062

The Teensy 4 (i.MXRT1062) controller driver (fork of `grblHAL/iMXRT1062`). Unlike the other two, the firmware changes live **in submodule forks**, not the driver repo — the driver carries the build config and the index. PRs: `core` #1–4 (combined littlefs+YModem, build-timestamp line, littlefs macro-search-path fix, ngc-flowctrl whitespace fix), `Plugin_SD_card` #1–2 (reset-during-SD-streaming, default-macro/ATC options), `Plugin_networking` #1 (hostname boot info). Build with PlatformIO (env `teensy41`). The `srw/local-build-config` branch pins all submodules to the fork branches for a turnkey build; `setup_forks.sh` wires the fork remotes.

> **Requirements & pairing.** The forks are built to run together; two headline features need their counterpart present:
>
> - **The 3D carve view (Simulator)** needs the bundled Fusion add-in `Fusion Addin/ioSenderBatchPost/` — it posts each CAM operation to a folder *and* writes a `0_ToolTable.nc` header carrying `(STOCK X=…)` + `(TOOL T=… D=…)` geometry. ioSender's *Load Folder* loads that first and forwards the comments to the simulator (the *send-comments* option forces them through), and the `sim-view` feature renders the carve from them. No add-in → no stock/tool geometry → the carve has nothing to draw. The add-in ships in the repo, so no external dependency.
> - **Running the stock upstream simulator instead of this fork?** ioSender still connects and streams g-code — homing, jogging and plain programs work — but every fork-only sim feature is absent: **no 3D carve view** (it doesn't exist upstream), no modeled probe surfaces (so the ATC Start-Job probe cycle can't be validated), no littlefs (no file-manager / macro-upload testing), and no settings persistence or `$REBOOT`.

3 · apply-prs — the composer

`apply-prs` (in `ioSender/tools/`) builds a fresh branch from a chosen set of PRs across any fork. It resolves each PR's dependency closure, merges them in topological order off the fork's base, and builds to verify. Append-only files (e.g. the locale CSVs) merge with a union driver so they never conflict; genuine same-line overlaps are reported up front and the merge aborts cleanly for a manual resolve. Forks are declared in `tools/forks.json` (repo path, base, manifest, build command, union globs).

```
# ioSender (the default fork)
python tools/apply-prs.py my-build 15 16 21      # compose (16 auto-pulls 15), build Relea
python tools/apply-prs.py --check 9 10           # just report same-line overlaps (no bran
python tools/apply-prs.py --list                 # list every PR + baseline/composable tag

# Simulator fork
python tools/apply-prs.py sim-all  --fork sim 1 2 3 4 5
python tools/apply-prs.py probe    --fork sim 2 --run   # compose, CMake build, launch grb
python tools/apply-prs.py --check  --fork sim 3 5       # overlap report for littlefs + si
```

```
# Simulator & firmware interlock — for USING them, take the whole bundle:
#   sim:      use the integration branch  (apply-prs --fork sim is a dev/review aid)
#   firmware: use srw/local-build-config  (turnkey, all submodule branches pinned)
#   their per-PR branches exist for upstream submission, not subset builds
```

Subset composition is really an **ioSender** concern — its ~18 features are independent, so picking a few is meaningful. The **Simulator** and **firmware** forks interlock and are few, so for *using* them you take the whole bundle (Simulator `integration`; firmware `srw/local-build-config`). Their granular per-PR branches exist for **upstream submission** (a maintainer may adopt just one), and `apply-prs --fork sim` is a build/review convenience rather than a consumer menu.

Each fork's `master` baseline differs: ioSender's already contains PRs 1–8 + 24 (requesting one just skips it as "already in master"); the Simulator's is pristine, so all five compose. Because a composed branch starts at the base, old branches that don't build standalone still build here.

4 · Effort & scale

Hours

Estimated from the git history (commit timestamps clustered into work sessions). The journey began **2026-05-30** with the very commit that started it — *"Fix build: add missing RP.Math references"* — and ran through 2026-06-23: **608 commits** over **25 active days**, an estimated **~110 hours** of focused work.

Week	Commits	~Hours	Focus
May 30 – 31	10	1.5	build fix (RP.Math) + first features
Jun 1 – 7	150	34	UI/jog, ATC, connect-simulator, first trackers
Jun 8 – 14	130	29	grbl:Settings, machine-setup, validate
Jun 15 – 21	151	38	ATC, localization, fork PRs, sim/firmware
Jun 22 – 23	167	11	localization sweep, trackers, apply-prs tooling
Total	**608**	**~113**	**~4.5 h/day on active days**

By area, the two largest buckets were **documentation/trackers** and **localization** — the polish weighed as much as any single feature. By repo: ioSender ~520 commits, Simulator ~58, firmware ~30.

This is an approximation from commit timestamps, not tracked time: it misses un-committed thinking/testing (under-counts) and credits whole windows during fast bursts (over-counts) — treat ~100–115 h as the honest band.

Tokens

Totalled from the local Claude Code session logs across both forks — **24,436** assistant turns:

Category	Tokens	
Fresh input	3.7 M	new prompt text each turn
Cache write	218.7 M	context written to cache
Cache read	10,571.9 M	context re-read each turn (the giant one)

Output (generated)	**48.5 M**	the code, docs & analysis the assistant wrote
Total processed	**~10.84 B**	~97% cached-context reads

Two figures are worth quoting: **~48 M output tokens** generated across ~24 K turns (what the assistant actually wrote), and **~10.8 B tokens processed** including the re-read conversation context. At Opus list API rates the equivalent metered cost would be ~$24 K — but on a flat-rate subscription that is notional, not what was paid. (Reproduce with `npx ccusage@latest daily`, which reads the same logs.)

Codebase size

Source lines in each fork vs its upstream baseline — the fork's own code only (excludes build output, vendored libraries and the shared grblHAL submodules):

Project	Upstream	Fork	Added	
ioSender (.cs + .xaml)	54,738	68,686	+13,948	+25%
Simulator (.c / .h)	3,199	5,301	+2,102	+66%
Firmware driver (.c / .h)	16,302	16,370	+68	surgical submodule patches

The firmware's substantive changes live in three submodule forks (`core`, `Plugin_SD_card`, `Plugin_networking`) as small focused PRs, so the driver itself barely moves — the firmware tracker carries the per-PR diffs.

5 · The proposal trackers

Each fork's full PR detail — features index, per-PR file/line counts, dependency matrix, and how-to-consume notes:

Fork	Tracker (HTML)	Rendered (PDF)
SENDER ioSender	ProposedPRs.html	ProposedPRs.pdf
SIM Simulator	ProposedPRs.html	ProposedPRs.pdf
FIRMWARE iMXRT1062	srw/ProposedPRs.html	srw/ProposedPRs.pdf

Tooling: `apply-prs.py` (composer) · `forks.json` (fork registry) · `prs.json` (ioSender manifest) · `Simulator/tools/prs.json` (sim manifest) · `firmware-forks.json` + `setup_forks.sh` (firmware index).